



**UNIVERSITÀ
DEGLI STUDI
DI TRIESTE**

Solving Chess Endgames

A Reinforcement Learning Approach

Valeria De Stasio, Christian Faccio, Giovanni Lucarelli

September 5, 2025

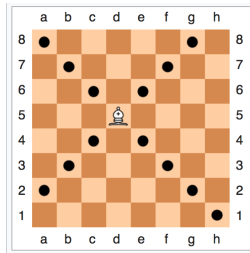
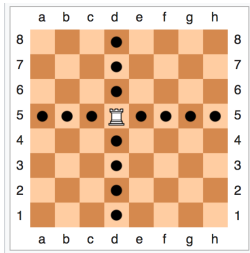
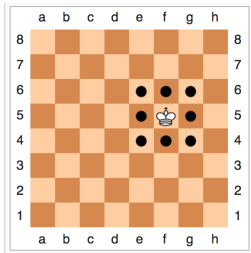
Objectives:

- Fastest checkmate against optimal player
- Experiment with training against random player
- Comparison of different RL approaches

Chess Programming Background

- 1890 El Ajedrecista: electro-mechanical KRK solver
- 1950 Claude Shannon: first article about chess programming
- 1997 *Deep Blue*: first computer program to beat the current world champion
- 2008 *Stockfish*: strong heuristic-based chess engine
- 2017 *Alphazero*: neural network-based approach that defeated Stockfish
- from 2018: more models developed based on *Alphazero*

Chess Rules

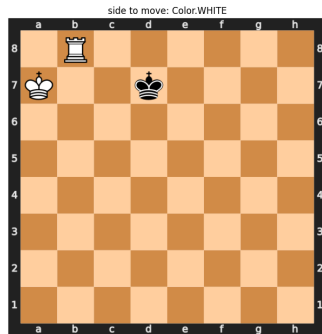


- **Win:** checkmate the opponent's king
- **Draw:** stalemate or insufficient material¹

¹threefold repetition and the fifty-move rule are not considered in our implementation

Elementary Endgames

- subset of endgames with few pieces left
- foundation for learning more complex endgames
- clear theoretical results: forced checkmate or draw
- Sygyzy endgame tablebases



Remark: white is always winning in elementary endgames!

MDP Formulation

Markov Game vs MDP

- Chess is a 2-player, turn-based, zero-sum game, formally a Markov Game
- Assume fixed black defender policy: **oracle** or **random**
- Black becomes part of the environment:
Markov Game $\rightarrow$ MDP

MDP: States

- State space $\mathcal{S}$: legal pieces positions and side-to-move

```
8/8/K7/8/8/5k2/7R/8 w - - 0 1
```

```
std::array<std::array<U64, 6>, 2> pieces;  
Color side_to_move;
```

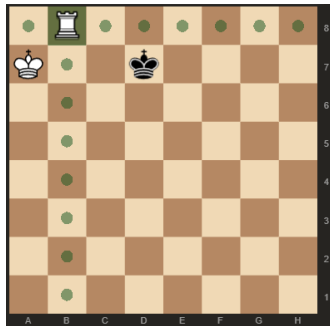
- Terminal state space $\bar{\mathcal{S}} \subset \mathcal{S}$: checkmate or draw positions (and stm). Defined procedurally

```
state.is_game_over() -> bool
```

MDP: Actions

- Action space $\mathcal{A}(s)$: set of legal moves for the current player.
- Defined procedurally:

```
state.legal_moves() ->  
std::vector<Move>
```



Transition probability $\Pr(S_{t+1} = s' | S_t = s, A_t = a)$

- **Oracle Defender:** deterministic evolution

```
state.do_move(Move)           // white deterministic
state.do_move(oracle.reply()) // black deterministic
```

- **Random Defender:** stochastic evolution

```
state.do_move(Move)           // white deterministic
state.do_move(random_reply()) // black random
```

Goal: find the fastest mate

Reward:²

- checkmate: +1
- draw: -1 (always winning for white)
- encourage faster mates
 - step penalty: -2/-0.01
 - discount factor $\gamma = 0.99$

²Except for Value Iteration

Choice of parameters

- **Step penalty:** $-2 \rightarrow$ values = DTZ
- **Draw penalty:** $-100 \rightarrow$ slow mates $>$ quick draws
- $\gamma = 1$
- $\theta = 0.0001$

Parameters Choice

- **Policy:** MLP(128,128,64), ReLU activations, softmax output
- **Value function:** $\hat{V}(s; w) = \sum_{i=1}^{16} w_i \phi_i(s)$
- $\gamma = 0.99$
- **Step penalty:** -0.01
- **Draw penalty:** -1

•

Monte Carlo Tree Search

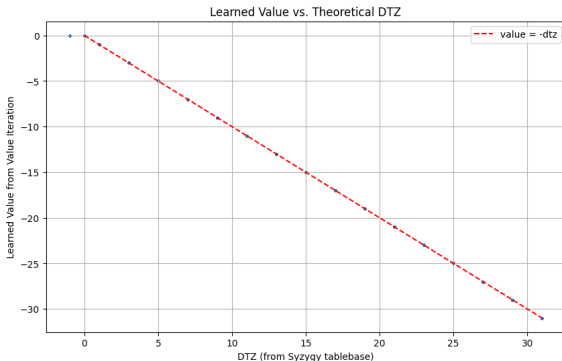
- One of the most common approaches to chess
- No need to train!
- **Step penalty:** 0
- **Draw penalty:** -1
- **N iterations:** 50000
- **Exploration constant:** $c = \sqrt{2}$
- **Upper confidence bound** applied to trees (UCT):

•

Results

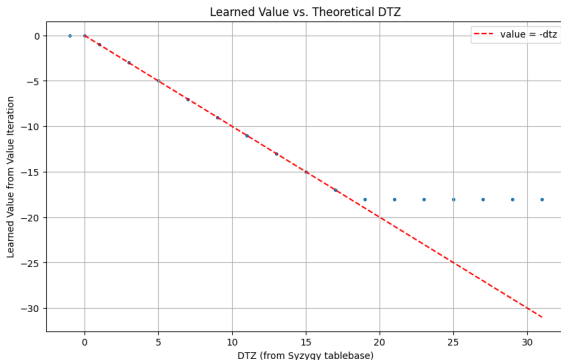
Value iterations

- Convergence to π_* after 17 iterations (~ 6 h wct on a laptop)
- Optimal policy π_* plays perfectly against an optimal opponent $\forall$ DTZ



Partial value iteration

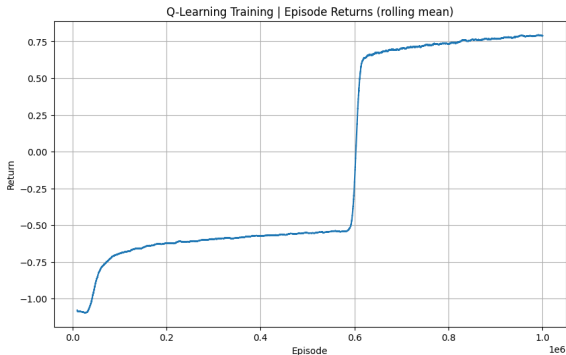
After n iterations it is able to solve mates only up to $DTZ = 2n + 1$



Here $n = 9$

Q-learning convergence

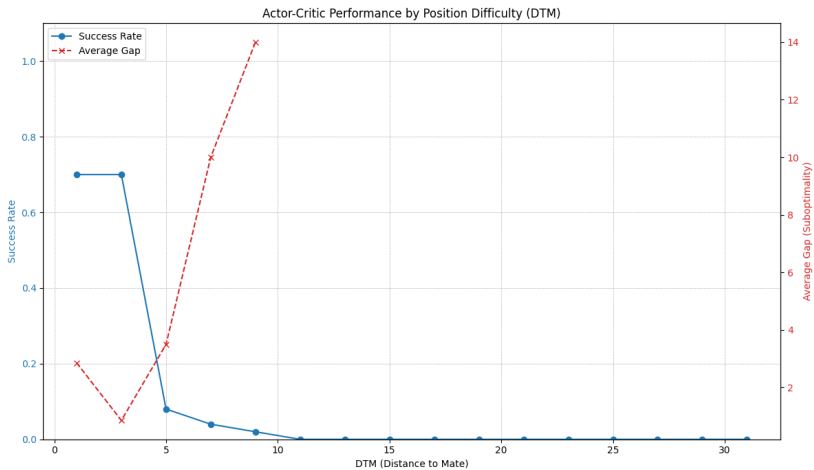
Convergence to **suboptimal** policy after 1M episodes
(~ 8h wct)





Actor-Critic convergence

- Partial convergence after 3k epochs (1.5k DTZ=1, 1.5k DTZ=3) ($\sim$ 6h wct)
- Suboptimal policy
- Explainable critic!



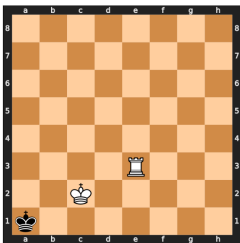
Critic through human chess principles

- $d_edgeBK = -0.269$
- $d_cornerBK = -0.402$
- $dInf(WK, BK) = -0.286$
- $dMan(WK, BK) = -0.375$
- $dInf(WR, BK) = -0.069$
- $dMan(WR, BK) = +0.200$
- $cut_w = +0.103$

With 10s of budget per move → solve mates up to DTZ=5

Top 5 white moves from root:

1.	e3a3	visits=	4897	pct=	97.9%	Q=+1.0000
2.	e3c3	visits=	10	pct=	0.2%	Q=-0.3562
3.	e3e4	visits=	8	pct=	0.2%	Q=-0.4388
4.	e3e2	visits=	5	pct=	0.1%	Q=-0.8457
5.	e3e7	visits=	5	pct=	0.1%	Q=-0.8516



Against the oracle ($DTZ = 1$)

Method	gap	success	ms_per_move	top1
VI	0.000	1.0	0.002	1.000
Q-Learning	4.600	1.0	0.002	0.823
Actor-Critic	2.857	0.7	0.865	0.639
MCTS	0.000	1.0	5155.985	1.000

Against the oracle (DTZ=3)

Method	gap	success	ms_per_move	top1
VI	0.000	1.0	0.003	1.000
Q-Learning	31.111	0.9	0.003	0.664
Actor-Critic	1.200	0.5	1.231	0.606
MCTS	0.000	1.0	7312.591	1.000

Against the Dummy (DTZ=1)

Method	gap	success	ms_per_move	top1
VI	0.000	1.0	0.004	1.000
Q-Learning	0.857	0.7	0.004	0.686
Actor-Critic	11.429	0.7	0.987	0.574
MCTS	0.000	1.0	8440.939	1.000

Against the Dummy (DTZ=3)

Method	gap	success	ms_per_move	top1
VI	0.000	1.0	0.002	1.000
Q-Learning	34.000	0.3	0.002	0.180
Actor-Critic	10.667	0.9	0.835	0.554
MCTS	2.400	1.0	8395.397	0.914

Why no convergence?

- Difficulties:
 - too many possible actions at each step
 - ideal opponent that never makes mistakes
 - very sparse positive rewards in training episodes
- Possible solutions:
 - more training time
 - implement draw by repetition and 50-move rule → more frequent terminal states

What did we learn?

- The importance of parameters choice (e.g., rewards)
- The danger of introducing a maximum length for each episode
- The importance of starting from simpler solutions and then increase complexity

Why is it so difficult to solve chess endgames?

A player can sometimes afford the luxury of an inaccurate move, or even a definite error, in the opening or middlegame without necessarily obtaining a lost position. In the endgame ... an error can be decisive, and we are rarely presented with a second chance.

- Paul Keres

Video Animation of optimal ply vs our policy

maybe for a mate in 5 or more

Future Work

- Zobrist hashing and invariance properties in chess endgames to reduce state space cardinality
- Implementing draw by repetition and 50-move rule
- Other chess endgames (e.g., KBBK, KRPK)
- Experiment with training against random player

Thank You!

Why $\gamma = 0.99$?

The longest rkr endgame requires 16 white moves, hence $\gamma^{16} \approx 0.85$ is a good reward for reaching the goal and producing fast mates.

